

PROJETO 2 - DESENHO DE ALGORITMOS

Routing Algorithm for Ocean Shipping and Urban Deliveries

Eduardo Cunha up202207126

Luna Cunha up202205714

Rodrigo Araújo up202205515

Interface do user

- permite ao usuário interagir com o sistema para resolver o Travelling Salesperson Problem (TSP) em diferentes tipos de grafos
- o user pode escolher entre 3 tipos de grafos:
 - toy graph
 - real world graph
 - extra medium-size graph
- quando seleciona um tipo de grafo, o user é direcionado para a seleção de um algoritmo para resolver o TSP
- possíveis algoritmos: “backtracking algorithm”, “triangular approximation” ou “other heuristics”

```
#####  
#      Select an option:      #  
#-----#  
#  1 -> Toy Graph            #  
#  2 -> Real World Graph      #  
#  3 -> Extra Medium-Size Graph #  
#  E -> EXIT                  #  
#                              #  
#####
```

```
#####  
#      Select an algorithm:    #  
#-----#  
#  1 -> Backtracking Algorithm #  
#  2 -> Triangular Approximation #  
#  3 -> Other Heuristics       #  
#  B -> BACK                    #  
#  E -> EXIT                    #  
#                              #  
#####
```

Interface do user - exemplo

```
#####
#      Select an option:      #
#-----#
#  1 -> Toy Graph            #
#  2 -> Real World Graph     #
#  3 -> Extra Medium-Size Graph #
#  E -> EXIT                 #
#                             #
#####
Option:
1
```

```
#####
#      Select an option:      #
#-----#
#  1 -> shipping             #
#  2 -> stadiums            #
#  3 -> tourism              #
#  B -> BACK                 #
#  E -> EXIT                 #
#                             #
#####
Option:
1
```

```
#####
#      Select an algorithm:    #
#-----#
#  1 -> Backtracking Algorithm #
#  2 -> Triangular Approximation #
#  3 -> Other Heuristics       #
#  B -> BACK                    #
#  E -> EXIT                    #
#                               #
#####
Option:
1
| The heuristic found a distance of 86.7 meters
| Total time elapsed: 0.065318 seconds
| The path is:
0 -> 1 -> 11 -> 10 -> 12 -> 13 -> 3 -> 2 -> 4 -> 6 -> 9 -> 7 -> 8 -> 5
```

Load and parse dataset files

- o parsing é responsável por carregar os dados dos grafos a partir de ficheiros csv e criar as estruturas de dados necessárias para os algoritmos do TSP
- criamos 3 funções para lidar com os diferentes tipos de grafos
 - **toyGraph** (exemplo do lado)
 - **realWorldGraph**
 - **extraGraph**
- depois da execução de cada uma destas funções, o grafo carregado é armazenado na variável finalGraph, sendo posteriormente usado nos algoritmos

```
GraphInterface *toyGraph(const string &filepath) {
    GraphInterface *graph = new GraphInterface();
    ifstream file(s: filepath);

    if (!file.is_open()) {
        cerr << "Error opening file: " << filepath << endl;
        return nullptr;
    }

    string line;
    getline(& file, & line); // Skip the header line

    while (getline(& file, & line)) {
        stringstream ss(s: line);
        string source, dest, distance;
        int num_columns = count(first: line.begin(), last: line.end(), value: ',') + 1;

        if (num_columns == 3) { // for shipping.csv and stadiums.csv
            getline(& ss, & source, dlm: ',');
            getline(& ss, & dest, dlm: ',');
            getline(& ss, & distance, dlm: ',');
        }
        else if (num_columns == 5) { // for tourism.csv
            getline(& ss, & source, dlm: ',');
            getline(& ss, & dest, dlm: ',');
            getline(& ss, & distance, dlm: ',');
            ss.ignore(n: numeric_limits<streamsize>::max(), dlm: ',');
            ss.ignore(n: numeric_limits<streamsize>::max(), dlm: ',');
        }
        else {
            cerr << "Invalid number of columns in CSV file: " << filepath << endl;
            return nullptr;
        }

        graph->addNode(index: stoi(str: source));
        graph->addNode(index: stoi(str: dest));
        graph->addBidirectionalEdge(node1: stoi(str: source), node2: stoi(str: dest), w: stod(str: distance));
    }

    finalGraph = graph;
    return finalGraph;
}
```


Backtracking Algorithm

backtrackTSP

- faz uma busca exaustiva para encontrar o caminho mais curto que visita todos os nós do grafo exatamente uma vez e retorna ao nó inicial
- explora todas as permutações de caminhos
- vai marcando os nós visitados e vai atualizando a menor distância quando encontra um novo caminho completo

```
// Backtracking algorithm for TSP
void GraphInterface::backtrackTSP(Node* current_node, unsigned int current_index, double total_distance,
                                  std::vector<Node*>& current_path, double& best_distance, std::vector<Node*>& best_path) {

    current_node->setVisited(true);
    current_path[current_index - 1] = current_node;

    if (current_index == nodes.size()) { // reached all nodes
        current_node->setVisited(false);
        // see if there is an edge back to the start node
        for (auto edge : Edge* : current_node->getAdj()) {
            if (edge->getDest()->getIndex() == 0) {
                double return_edge_weight = edge->getWeight();
                double total = total_distance + return_edge_weight;
                if (total < best_distance) {
                    best_distance = total;
                    best_path = current_path;
                }
                break;
            }
        }
        return;
    }

    for (auto edge : Edge* : current_node->getAdj()) {
        Node* next_node = edge->getDest();
        double edge_weight = edge->getWeight();
        if (!next_node->isVisited() && total_distance + edge_weight < best_distance) {
            backtrackTSP(current_node: next_node, current_index: current_index + 1,
                        total_distance: total_distance + edge_weight, &: current_path, &: best_distance, &: best_path);
        }
    }

    current_node->setVisited(false);
}
```

Backtracking Algorithm

solveTSPBacktracking

- inicializa variáveis e configura o ambiente para o algoritmo de backtracking
- armazena o caminho atual num vetor e define uma menor distância inicial
- marca todos os nós como unvisited e começa a busca ao chamar a função backtrackTSP a partir do 1º vértice
- retorna a menor distância encontrada

```
// Solve TSP using backtracking
double GraphInterface::solveTSPBacktracking(std::vector<Node*>& path) {
    std::vector<Node*> current_path(n: nodes.size());
    double shortest_distance = DBL_MAX;

    for (auto node : Node* : nodes) {
        node->setVisited(false);
    }

    backtrackTSP(current_node: nodes[0], current_index: 1, total_distance: 0.0,
        &: current_path, &: shortest_distance, &: path);

    return shortest_distance;
}
```

Triangular Approximation Heuristic

toRadians

- converte graus para radianos
- é usada na função haversine para cálculos trigonométricos em coordenadas geográficas

haversine

- calcula a distância entre dois pontos da superfície da Terra (neste caso)
- recebe as coordenadas da latitude e da longitude de dois pontos e retorna a distância aproximada em metros

```
// Convert degrees to radians
double GraphInterface::toRadians(double degrees) {
    return degrees * M_PI / 180.0;
}

// Haversine formula to calculate distance between two points
double GraphInterface::haversine(double lat1, double lon1, double lat2, double lon2) {
    double earth_radius = 6371000; // meters

    double rad_lat1 = toRadians(degrees: lat1);
    double rad_lon1 = toRadians(degrees: lon1);
    double rad_lat2 = toRadians(degrees: lat2);
    double rad_lon2 = toRadians(degrees: lon2);

    double delta_lat = rad_lat2 - rad_lat1;
    double delta_lon = rad_lon2 - rad_lon1;

    double aux = sin(delta_lat / 2) * sin(delta_lat / 2) +
                cos(rad_lat1) * cos(rad_lat2) *
                sin(delta_lon / 2) * sin(delta_lon / 2);

    double c = 2.0 * atan2(sqrt(aux), sqrt(1.0 - aux));

    return earth_radius * c;
}
```


Triangular Approximation Heuristic

triangularApproximationHeuristic

- começa num vértice inicial e vai visitando os outros, escolhendo sempre o vizinho mais próximo disponível
- repete isto até todos os vértices terem sido visitados e, quando não há mais nós por visitar, volta ao inicial para completar o ciclo
- retorna a distância total percorrida que é calculada ao longo do processo

```
double GraphInterface::triangularApproximationHeuristic(std::vector<Node*>& path) {  
    double approx_distance = 0.0;  
  
    Node* current_node = findNode(index: 0);  
    current_node->setVisited(true);  
    path.push_back(current_node);  
  
    while (path.size() < nodes.size()) {  
        double min_distance = DBL_MAX;  
        Node* next_node = nullptr;  
  
        for (auto edge : Edge* : current_node->getAdj()) {  
            Node* neighbor_node = edge->getDest();  
            if (!neighbor_node->isVisited()) {  
                double distance = edge->getWeight();  
                if (distance < min_distance) {  
                    min_distance = distance;  
                    next_node = neighbor_node;  
                }  
            }  
        }  
  
        if (next_node != nullptr) {  
            approx_distance += min_distance;  
            next_node->setVisited(true);  
            path.push_back(next_node);  
            current_node = next_node;  
        }  
        else {  
            break;  
        }  
    }  
  
    for (auto edge : Edge* : current_node->getAdj()) {  
        if (edge->getDest()->getIndex() == 0) {  
            approx_distance += edge->getWeight();  
            path.push_back(findNode(index: 0));  
            break;  
        }  
    }  
  
    return approx_distance;  
}
```


Other Heuristics

nearestNeighborHeuristic

- começa num vértice inicial e, em cada etapa, escolhe o nó adjacente não visitado mais próximo para visitar a seguir
- repete isto até todos os vértices terem sido visitados e, quando depois de visitar o último nó, volta ao inicial para completar o ciclo
- calcula a distância total e chama a função twoOptOptimization para melhorar a qualidade da solução inicial
- retorna a distância total otimizada

```
double GraphInterface::nearestNeighborHeuristic(std::vector<Node*>& path) {
    double total_distance = 0.0;
    std::vector<bool> visited(n: nodes.size(), x: false);

    Node* current_node = nodes[0];
    path.push_back(current_node);
    visited[current_node->getIndex()] = true;

    while (path.size() < nodes.size()) {
        Node* next_node = nullptr;
        double shortest_distance = DBL_MAX;

        for (auto edge: Edge* : current_node->getAdj()) {
            Node* neighbor = edge->getDest();
            if (!visited[neighbor->getIndex()]) {
                double dist = edge->getWeight();
                if (dist < shortest_distance) {
                    shortest_distance = dist;
                    next_node = neighbor;
                }
            }
        }

        if (next_node) {
            total_distance += shortest_distance;
            current_node = next_node;
            path.push_back(current_node);
            visited[current_node->getIndex()] = true;
        } else {
            break;
        }
    }

    total_distance += haversine(lat1: current_node->getLatitude(), lon1: current_node->getLongitude(),
                                lat2: nodes[0]->getLatitude(), lon2: nodes[0]->getLongitude());
    path.push_back(nodes[0]);

    total_distance = twoOptOptimization(& path, current_distance: total_distance);

    return total_distance;
}
```

Other Heuristics

twoOptOptimization

- verifica todas as possíveis trocas de dois segmentos do caminho para ver se alguma dessas trocas conseguiria reduzir a distância total
- se for encontrada uma melhoria, os segmentos são invertidos e calcula-se a nova distância
- o processo continua até deixar de ser possível fazer alguma melhoria adicional
- retorna a distância total otimizada

```
double GraphInterface::twoOptOptimization(std::vector<Node*>& path, double current_distance) {
    bool improvement = true;

    while (improvement) {
        improvement = false;
        for (size_t i = 1; i < path.size() - 2; i++) {
            for (size_t j = i + 1; j < path.size() - 1; j++) {
                double new_distance = current_distance -
                    haversine( lat1: path[i-1]->getLatitude(), lon1: path[i-1]->getLongitude(),
                              lat2: path[i]->getLatitude(), lon2: path[i]->getLongitude()) -
                    haversine( lat1: path[j]->getLatitude(), lon1: path[j]->getLongitude(),
                              lat2: path[j+1]->getLatitude(), lon2: path[j+1]->getLongitude()) +
                    haversine( lat1: path[i-1]->getLatitude(), lon1: path[i-1]->getLongitude(),
                              lat2: path[j]->getLatitude(), lon2: path[j]->getLongitude()) +
                    haversine( lat1: path[i]->getLatitude(), lon1: path[i]->getLongitude(),
                              lat2: path[j+1]->getLatitude(), lon2: path[j+1]->getLongitude());

                if (new_distance < current_distance) {
                    std::reverse( first: path.begin() + i, last: path.begin() + j + 1);
                    current_distance = new_distance;
                    improvement = true;
                }
            }
        }
    }

    return current_distance;
}
```

Eficiência e Optimalidade

vantagens e desvantagens de cada
algoritmo

Backtracking

- é ineficiente para grafos grandes
- garante solução ótima
- viável para grafos pequenos

Triangular Approximation Heuristic

- eficiente e prática para grafos de tamanho médio
- pode resultar em soluções subótimas

Other Heuristics

- muito eficiente e fácil de implementar
- adequada para soluções rápidas em grafos de tamanho médio a grande
- não garante solução ótima